NotFlex: Compilador de Expresiones Regulares a Código C Optimizado para Entornos Embebidos

Hillebrand Giuliano, Sienra Daiana y González Martín

Universidad Gastón Dachary, Posadas, Misiones, Argentina

Resumen NotFlex es un compilador que toma una expresión regular y genera un autómata finito determinista y código C (estándar C99) listo para ser utilzado en sistemas embebidos e IoT. El código C generado es 2,2 y 11,5 veces más rápido que usar `regex.h` de POSIX dado que no utiliza memoria dinámica ni librerías externas. Además, el código C generado puede recorrer una cadena en tiempo lineal $\mathcal{O}(n)$. El compilador se divide en frontend y backend. En el frontend donde se realiza el análisis léxico, sintáctico, semántico y donde se arma el Árbol de Sintaxis Exacta; y en el backend el motor de autómatas aplica la construcción de Thompson, subconjuntos que agrupan caracteres en clases de equivalencia y minimización de Hopcroft.

Palabras clave: Compiladores · Expresiones Regulares · Autómatas Finitos · Sistemas Embebidos · Minimización de Hopcroft · Código C Optimizado.

1. Introducción

El foco de nuestro proyecto son los sistemas embebidos e IoT (Internet de las Cosas). En este contexto, la memoria y la capacidad de procesamiento son recursos limitados, y en ocasiones se necesita validar cadenas de texto, como tramas obtenidas por sensores, identificadores o comando por puerto serie. Para abordar estos problemas existen dos maneras de hacerlo, pero ambas presentan sus desventajas:

- Escribir un validador a mano, a base de `if/switch` anidados que pueden ser complejos y donde es fácil equivocarse.
- Implementar una librería de expresiones regulares, como `regex.h` de POSIX que ocupan mucho de estos escasos recursos recién mencionados, porque incluso utilizan asignación de memoria dinámica. Además, si la librería implementada utiliza backtracking, como PCRE2, podría llegar a tardar un tiempo exponencial al utilizarse con ciertos patrones.

Como solución ante estas dificultades, proponemos NotFlex. Un compilador que hace el trabajo pesado en tiempo de compilación y dada una expresión regular de entrada, genera como salida una función C en el estándar C99 muy ligera que recorre una tabla de transiciones estática, por lo que no utiliza asignación

dinámica de memoria y permite dedicar los limitados recursos a otras necesidades, ya que no depende de librerías adicionales ni de un motor de expresiones regulares.

2. Metodología de Desarrollo

NotFlex se desarrolló de forma asistida por inteligencia artificial (IA). Implementamos un enfoque de Desarrollo Dirigido por Especificaciones y una suite de tests que nos permitió auditar el funcionamiento del código generado por IA para luego ser integrado oficialmente al proyecto. Esto NO quiere decir que el trabajo es una caja negra, realizado en su totalidad por IA a través de un "Claude, hacelo, y no cometas errores". Los integrantes del equipo estuvimos muy involucrados en el desarrollo del proyecto y vale destacar que este informe es 100 % de nuestra autoría y 0 % hecho con IA. Podemos resumir nuestro flujo de trabajo en *el equipo investiga y especifica, luego la IA implementa, el equipo testea y audita para que finalmente el equipo integre el desarrollo al proyecto entregable.*

2.1. De la investigación, al informe y luego a la especificación

A medida que se desarrollaba el cursado de la cátedra de Compiladores, el equipo continuamente detallaba este informe con las necesidades y limitaciones que nuestro trabajo integrador debía cumplir. Con lo visto en la cátedra (las fases clásicas de un compilador) definimos la idea, el objetivo, el alcance, las transformaciones del motor y los escenarios de prueba que demuestran que el objetivo se cumple. Y a partir de este documento derivamos tareas acotadas y precisas (por ejemplo, el analizador léxico, las definiciones con tabla de símbolos, el motor de autómatas), acompañadas de sus requisitos y su criterio de aceptación que luego entregamos a la IA como especificación.

Las IAs utilizadas fueron:

- Gemini 4 Argón: modelo de inteligencia artificial de frontera desarrollado por Google DeepMind, diseñado específicamente para ofrecer un razonamiento profundo dinámico, inteligencia de alto nivel y el máximo rendimiento en flujos de trabajo complejos y exigentes.
- Claude Opus 5.5: modelo insignia de Anthropic diseñado específicamente para el trabajo de conocimiento de alta complejidad y el desarrollo de software agéntico de larga duración.

2.2. Los tests como mecanismo de auditoría

Para establecer una tarea dentro del proyecto, el criterio de aceptación de cada una de ellas se expresó como pruebas automáticas implementadas con `pytest`. El equipo fue definiendo los tests iterativamente a medida que el desarrollo del proyecto progresaba, al final del desarrollo, La suite completa cuenta con 268 pruebas y combina cuatro tipos:

- **Unitarias y de integración:** Son unitarios y de integracón porque verifican que cada pieza o parte haga lo que se supone que debe hacer, por si misma (unitaria) o conectada a otras (integración). Detectan errores de lógica local como un mensaje de error en una posición equivocada pero están limitados a casos que el equipo pensó de antemano y no pueden testear lo que a nadie se le ocurrió.
- **De caracterización:** Fueron escritos sobre el código existente para establecer lo que el código hacía en un momento determinado y nos permitieron modificar el código existente sin accidentalmente romper algo. Su objetivo es detectar regresiones, por ejemplo, si en algún momento alguien cambiaba accidentalmente como se tokenizaba el caracter
  el test fallaba. También están limitados a que si accidentalmente se definía un test y se configuraba erroneamente, el test podría dar como válido un error.
- **Diferenciales:** Nos permiten comparar la salida del compilador con la de una implementación independiente. Por ejemplo, hay un test que se encarga de traducir el AST a una expresión de la librería re y verifica 12 ejemplos predefinidos con cadenas aceptadas y rechazadas.
- **De propiedades no funcionales:** Este tipo de test no verificaba el correcto funcionamiento del código, si no que nos permitieron validar nuestro objetivo de generar código C ligero y muy optimizado para entornos con recursos limitados. Este tipo de tests nos permitió realizar las afirmaciones en la Sección 8.

3. Arquitectura General del Compilador

NotFlex se basa en las etapas tradicionales de un compilador y en las mismas bases teóricas que Flex (Thompson, subconjuntos, clases de equivalencia). Por lo que el pipeline de compilación de NotFlex sigue una serie de pasos secuenciales y conectados, clásicos de un compilador, dividido en tres etapas (Fig. 1):

1. **Frontend (Análisis):** El analizador léxico separa la expresión en tokens, el parser descendente recursivo arma el AST respetando la precedencia de los operadores, y el análisis semántico resuelve las definiciones con nombre usando una tabla de símbolos.
2. **Motor de Autómatas (Síntesis):** Pasa el AST a un AFND con Thompson y lo convierte en AFD con subconjuntos, agrupando en el mismo paso los 256 valores posibles de un byte en clases de equivalencia. Por último aplica Hopcroft, un método que reduce al mínimo el número de estados del AFD.
3. **Backend (Generación de Código):** Genera el archivo C en el estándar C99 crenado un `.c` y un `.h` con la tabla de transiciones y el bucle de validación.

4. Especificación Sintáctica y Léxica

Nuestro proyecto acepta expresiones regulares, es decir, lenguajes de Tipo 3 en la jerarquía de Chomsky. Decidimos acotar la sintaxis de las expresiones

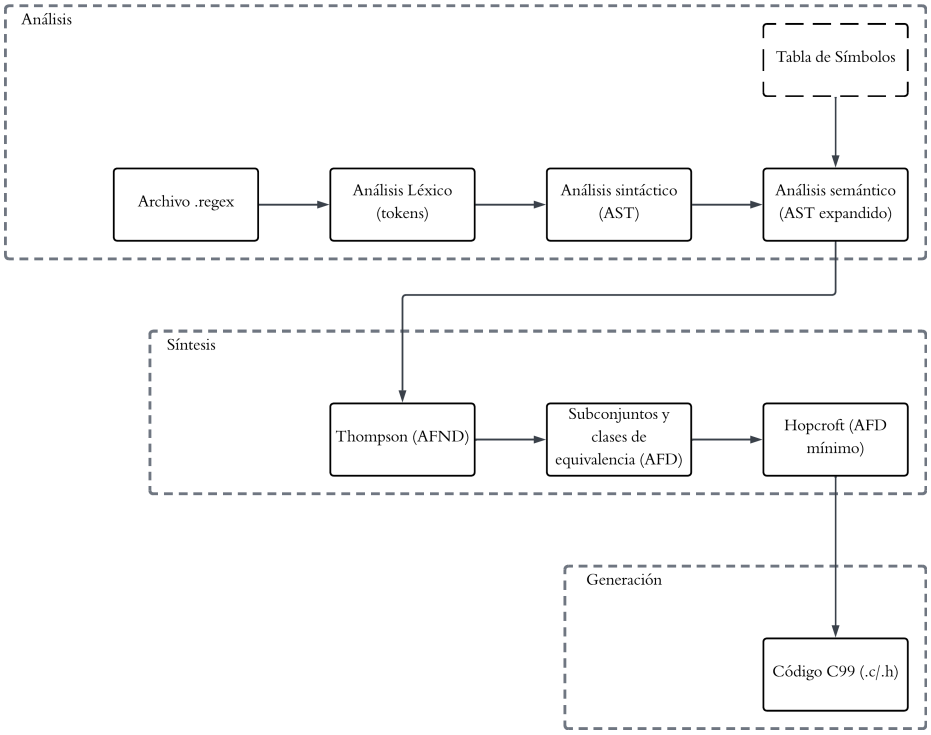


Figura 1. Pasos secuenciales de compilación de NotFlex en tres etapas

regulares de entrada a las siguientes expresiones con el objetivo de no sobrecargar el proyecto:

- **Literales y Concatenación:** `ab`
- **Alternancia (Unión):** `a|b`
- **Clausuras:** Kleene (`a*`), Positiva (`a+`) y Opción (`a?`)
- **Agrupamiento y Clases:** `(a|b)*`, `[a-zA-Z0-9_]`
- **Comodín:** `.` coincide con cualquier carácter salvo el salto de línea.
- **Escapes de caracteres:** `\n`, `\t` y una barra seguida de un símbolo, que lo vuelve literal (`\+`, `\.`).
- **Referencias:** `{NOMBRE}` usa una definición con nombre (ver Sección 4.2).

4.1. Gramática

Para el análisis sintáctico decidimos implementar un parser descendente recursivo, con una función por cada no terminal, que sigue la siguiente grmática:

$$
\begin{aligned}
\mathit{expr} &\rightarrow \mathit{subexpr}\ (\texttt{|}\ \mathit{subexpr})^* \\
\mathit{subexpr} &\rightarrow \mathit{factor}\ \mathit{factor}^* \\
\mathit{factor} &\rightarrow \mathit{átomo}\ (\texttt{*} \mid \texttt{+} \mid \texttt{?})^* \\
\mathit{átomo} &\rightarrow \texttt{CHAR} \mid \texttt{CLASS} \mid \texttt{REF} \mid \texttt{(}\ \mathit{expr}\ \texttt{)}
\end{aligned}
$$

En cuanto a la precedencia, tenemos, de mayor a menor, los cuantificadores, la concatenación y la unión. Ademas, el conjunto PRIMERO de todos los no terminales es {`CHAR`, `CLASS`, `REF`, `(`}, y en cada repetición los tokens que la continúan son disjuntos de su conjunto SIGUIENTE. Por eso podemos afirmar que la gramática es LL(1). Por último, nuestro analizador léxico agrega un token de fin de entrada que cumple el rol del símbolo $.

4.2. Análisis semántico y tabla de símbolos

Para poder darle nombre a las subexpresiones, nuestro compilador admite definiciones al estilo de Flex, que se separan del patrón principal por `%%`, a continuacion mostramos un ejemplo:

```
DIGITO = [0-9]
ENTERO = {DIGITO}+
NUMERO = {ENTERO}(\.{ENTERO})?
TEMPERATURA = TEMP={NUMERO}
HUMEDAD = HUM={ENTERO}
%%
{TEMPERATURA};{HUMEDAD}
```

En la fase semántica registramos cada definición en la tabla de símbolos y al mismo tiempo verificamos que no haya errores. Rechazamos los siguientes casos:

- Las definiciones duplicadas.
- Las referencias sin definición.
- Las referencias circulares, en las que informamos el camino completo, por ejemplo `A -> B -> A`.
- Los caracteres fuera del alfabeto de 256 símbolos.

Por otro lado, si hay definiciones que nunca se usan no las consideramos un error, sino que generamos una advertencia. Como resultado de esta fase obtenemos el AST del patrón principal con todas las referencias ya expandidas que luego pasa a ser la entrada del motor del autómata.

5. Limitaciones y Trabajo Futuro

Limitaciones: Nuestro compilador solo sirve para patrones que se conocen al momento de compilación. Además, decidimos no soportar, con el objetivo de no sobrecargar el proyecto, los siguientes componentes de una expresión regular:

- La negación de clases (`[^a-z]`).
- Los cuantificadores acotados (`a{3,5}`).
- Las anclas (`^`, `$`).
- Las clases abreviadas (`\d`, `\w`).
- Las retroreferencias (backreference).

Cuando aparece alguna de estas construcciones, el compilador produce un error léxico en lugar de interpretarla como un literal, para que el comportamiento coincida con el del módulo re de Python.

6. Transformaciones e Implementación del Motor

Nuestro motor de autómatas aplica tres transformaciones, una después de la otra. Si sumamos la construcción del AST y la generación de código C, el pipeline completo tiene cinco etapas.

1. **Construcción de Thompson:** Recorremos el AST de forma recursiva y armamos el AFND, uniendo los distintos fragmentos con transiciones épsilon (ϵ).
2. **Construcción de Subconjuntos con Clases de Equivalencia:** Antes de construir el AFD, agrupamos los caracteres que se comportan igual en todo el autómata. Por ejemplo, para un identificador de C (`[a-zA-Z_][a-zA-Z0-9_]*`) los 256 valores posibles de un byte quedan reducidos a solo 3 clases (letra, dígito y otro). Después eliminamos el no determinismo calculando las ϵ-clausuras, pero recorriendo esas pocas clases en lugar de los 256 caracteres, lo que nos da una tabla de transiciones más chica.
3. **Minimización de Hopcroft:** Juntamos los estados que no se pueden distinguir entre sí. Cada estado que eliminamos achica el arreglo que se genera al final.

7. Generación de Código C (Backend)

En el backend se genera el código C en el estándar C99, la tabla de transiciones y el recorrido de la cadena. A modo de ejemplo, para un identificador el resultado podría verse así:

```c
/* Mapeo de los 256 valores de un byte a 3 clases de equivalencia */
static const unsigned char clase[256] = { /* generado automáticamente */ };

/* Tabla de transiciones: [estado][clase] */
static const unsigned char trans[3][3] = {
    /*          letra  digito  otro */
    /* q0 */ {    1,     2,     2 },  /* q0: estado inicial */
    /* q1 */ {    1,     1,     2 },  /* q1: estado de aceptación */
    /* q2 */ {    2,     2,     2 }   /* q2: estado de rechazo */
};

int validar_id(const char *s) {
    int estado = 0;
    while (*s != '\0') {
        estado = trans[estado][clase[(unsigned char)*s++]];
    }
    return estado == 1;
}
```

Como se puede ver la función no usa `malloc` (asignación de memoria dinámica), no depende de nada externo y hace un solo acceso a la matriz por cada carácter de la cadena.

8. Escenarios de Prueba y Evaluación

Para comprobar que NotFlex cumple con nuestros objetivos de ser eficiente en la utilización de recursos escasos se preparó una suite de pruebas que se pueden

repetir. Todas forman parte de la suite de `pytest` y las dos últimas también se pueden correr con `make valgrind` (para evaluar el uso de memoria dinámica) y `make bench` (para comparar velocidad contra regex.h de POSIX):

1. **Pruebas Unitarias:** Cubrimos cada fase del compilador (léxico, sintáctico y semántico), los mensajes de error con su posición, la línea de comandos y cada transformación del motor. Por ejemplo, verificamos que el identificador de C genere exactamente 3 clases y la tabla de 3×3 de la Sección 7). También probamos los doce ejemplos incluidos (IPv4, URL, correo, fecha, UUID y trama de sensor, entre otros) con cadenas que deben aceptarse y otras que deben rechazarse.
2. **Validación Lógica:** Hacemos fuzzing, esto significa que comparamos los resultados del AFD mínimo con los del módulo re de Python en 1500 cadenas aleatorias, que incluyen casos borde como la cadena vacía, saltos de línea y tabulaciones. Además, compilamos con gcc el código C generado y lo probamos directamente.
3. **Consumo de Memoria:** Compilamos el código generado de cuatro ejemplos junto con un programa de prueba y se lo ejecuta con Valgrind. En todos los casos el resumen informa `0 allocs, 0 frees, 0 bytes allocated`, sin errores de memoria. Ese programa de prueba no usa `stdio`, porque la biblioteca de C reserva en el *heap* el buffer de la salida estándar en la primera escritura, y Valgrind contaría esa reserva aunque la función generada no haga ninguna.
4. **Desempeño Comparativo:** traducimos el AST de cada ejemplo a una expresión regular extendida de POSIX y medimos el tiempo medio de 10^6 validaciones con nuestra función generada y con `regexec()` de `regex.h`, sin contar la compilación de la expresión (`regcomp()`). Antes de medir comprobamos que ambas clasifiquen igual 1000 cadenas por ejemplo (muestras y mutaciones aleatorias): no hubo ninguna discrepancia. La Tabla 1 muestra los resultados.

Tabla 1. Tiempo medio por cadena con 10^6 validaciones (`gcc -O2`, x86-64). Los valores absolutos dependen de la máquina.

Ejemplo	**NotFlex**	`regex.h`	**Aceleración**
Identificador de C	6,7 ns	76,5 ns	11,5×
IPv4	7,4 ns	62,1 ns	8,4×
Correo electrónico	15,5 ns	90,9 ns	5,9×
URL	35,8 ns	134,6 ns	3,8×
UUID	38,7 ns	83,3 ns	2,2×

La función generada fue entre 2,2 y 11,5 veces más rápida. La ventaja es mayor en los ejemplos de cadenas cortas (identificador, IPv4), esto sugiere que ahí pesa más el costo fijo de cada llamada a `regexec()`; y en las cadenas largas domina el recorrido carácter por carácter de ambos.

9. Comparación con Trabajos Relacionados

Nosotros no fuimos los primeros en pensar en esta idea, por lo que es razonable comparar nuestro proyecto con aquellas soluciones existentes. Existen librerías como `regex.h` o `PCRE2` que trabajan en tiempo de ejecución, pero a diferencia de éstas, NotFlex se inspira más en otras soluciones como `re2c` y `Ragel` que funcionan en tiempo de compilación. (Tabla 2). Anteriormente mencionamos que NotFlex está inspirado en Flex dado que se basa en la misma base teórica, pero no hay que confundirse, no hacen lo mismo, por eso lo llamamos "NotFlex":

- Flex: toma muchas reglas y produce un scanner que reconoce tokens dentro de un flujo de texto y,
- NotFlex: toma un patrón y produce una función C99 mínima que valida una cadena completa, sin memoria dinámica ni librerías, pensada para sistemas embebidos.

Y recién mencionamos `re2c` y `Ragel`, y si uno conoce estas herramientas, sabrá que ámbas sí generan código C sin librerías de ejecución, y hasta soporta más lenguaje de salida. ¿Entonces que nos diferencia de estas herramientas? Quizás la respuesta sea un poco decepcionante, pero es realista, ya que re2c y Ragel generan código equivalente y son herramientas maduras; no pretendemos superarlas. Nuestro aporte es implementar y documentar el pipeline completo, con una interfaz más directa (de un patrón a una función lista) y una metodología de desarrollo auditada con tests.

Tabla 2. Comparación de NotFlex frente a las alternativas tradicionales.

Característica	**Validador a Mano**	**Librería (`regex.h`)**	**NotFlex**
Dependencias externas	No	Sí	**No**
Memoria dinámica	No	Sí	**No (0 bytes)**
Tiempo de ejecución	Variable	Depende de la implementación	**Lineal** $O(n)$
Mantenibilidad	Dificultosa	Alta	**Alta**

10. Conclusiones

NotFlex logró lo que buscábamos: una serie de pasos automatizados que se ejecutan de forma encadenada y van de la expresión regular al código C. Integramos las fases clásicas de un compilador (análisis léxico, sintáctico y semántico con tabla de símbolos, transformaciones de autómatas, minimización y generación de C). El resultado es una función en C que no usa memoria dinámica y siempre recorre la cadena en tiempo lineal, ideal para dispositivos con recursos

limitados. Las pruebas nos respaldan, ya que Valgrind no registra ninguna reserva en el *heap*, y la función generada coincide con `regex.h` en todas las cadenas probadas y es entre 2,2 y 11,5 veces más rápida.

Glosario

AFD Autómata Finito Determinista. En cada estado, para cada carácter, hay un solo camino posible.

AFND Autómata Finito No Determinista. Puede tener varios caminos para un mismo carácter y transiciones épsilon.

Análisis semántico Fase que verifica el significado de la entrada, por ejemplo que cada referencia tenga una definición y que no haya definiciones circulares.

AST Árbol de Sintaxis Abstracta. Árbol que representa la estructura de la expresión regular después de analizarla.

Backtracking Técnica que prueba un camino y, si falla, vuelve atrás a probar otro. Con ciertos patrones puede tardar muchísimo.

C99 Versión del estándar del lenguaje C publicada en 1999. Se optó por usar esta porque casi todos los compiladores de microcontroladores la soportan.

Clase de equivalencia Grupo de caracteres que tienen las mismas transiciones en el autómata. Permite achicar la tabla.

Cota Límite máximo. Si el tiempo de ejecución tiene cota, se asegura que nunca pasa de cierto valor.

DSL *Domain-Specific Language*. Lenguaje o herramienta hecha para resolver un solo tipo de problema, en este caso expresiones regulares.

Embebido Sistema informático chico que está dentro de otro dispositivo y hace una tarea puntual, con poca memoria y poco procesamiento (por ejemplo, un sensor o un microondas).

ϵ-clausura Conjunto de estados a los que se llega desde un estado usando solamente transiciones épsilon (sin leer ningún carácter).

Fuzzing Prueba que consiste en generar muchísimas entradas (incluso raras) y comparar los resultados contra una referencia para encontrar errores.

Hopcroft Algoritmo que minimiza un AFD juntando los estados que no se pueden distinguir entre sí. El resultado es el AFD con menos estados posible.

Host La PC donde se ejecuta el compilador, a diferencia del dispositivo donde después corre el código generado.

IoT *Internet of Things* (Internet de las Cosas). Objetos cotidianos con sensores que se conectan a una red.

LL(1) Gramática que puede analizarse de izquierda a derecha decidiendo cada paso con un solo token de anticipación, lo que permite un parser descendente recursivo sin retroceso.

MVP *Minimum Viable Product*. Primera versión funcional con lo mínimo necesario.

$O(n)$ Tiempo lineal. El tiempo crece en proporción al largo n de la cadena: si la cadena es el doble de larga, tarda el doble.

Parser Parte del compilador que analiza el texto de la expresión regular y arma el AST.

Pipeline Serie de pasos encadenados donde la salida de uno es la entrada del siguiente.

Subconjuntos Algoritmo que convierte un AFND en un AFD. Cada estado del AFD representa un conjunto de estados del AFND.

Tabla de símbolos Estructura que guarda cada nombre definido en la entrada junto con su información (aquí, su AST, su línea y la cantidad de usos).

Thompson Algoritmo que convierte un AST en un AFND, armándolo por partes y uniéndolas con transiciones épsilon.

Valgrind Herramienta que ejecuta un programa y detecta errores de memoria. La usamos para comprobar que no hay reservas dinámicas (`malloc`).

Referencias

1. Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D.: Compilers: Principles, Techniques, and Tools. Pearson Education, 2nd edn. (2007)
2. Hopcroft, J.E., Motwani, R., Ullman, J.D.: Introduction to Automata Theory, Languages, and Computation. Addison-Wesley, 3rd edn. (2006)